

Project Design Documentation

Specialty Coffee Online Shop | Microservices and Containers (KM00DT91)

This document gives the high-level design of the coffee shop application built for the course project, using domain-driven design (DDD). It describes the domain breakdown, the microservices architecture (frontend, backend, database), the workflow pattern, and the part implemented in the prototype.

1. Domain-Driven Design Breakdown

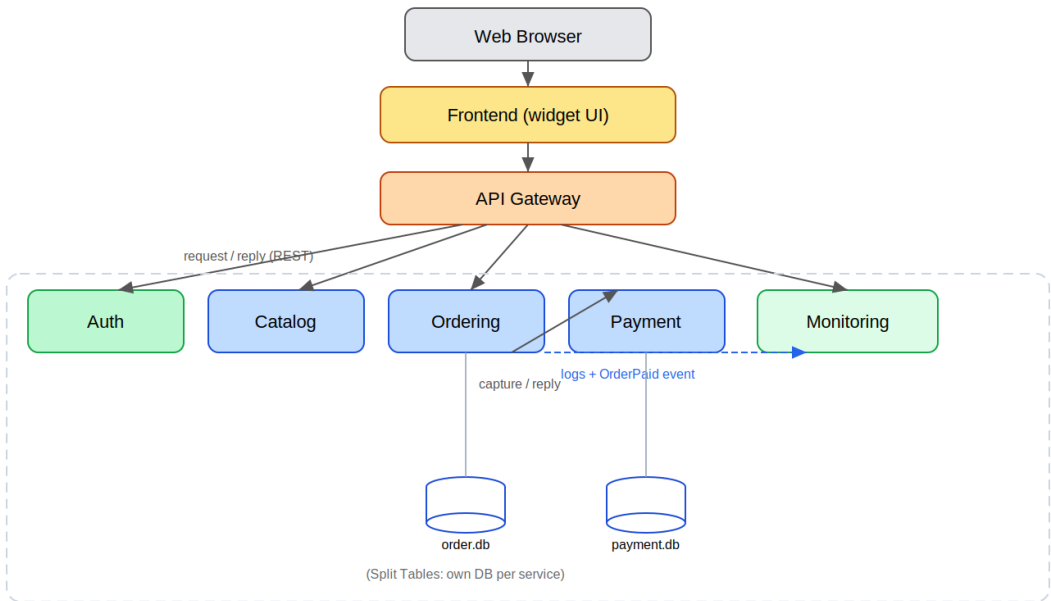
Domain: online retail of specialty coffee. A customer logs in, browses products, places an order, and the order is paid and recorded. The domain is split into subdomains, each owning a bounded context with its own aggregates; each bounded context is a candidate microservice that owns its own data.

Subdomain	Bounded context (service)	Aggregates	Main capabilities
Catalog	Product Catalog	Product, Category	List products (read), request-reply
Sales	Ordering	Order, Cart, OrderLine	Place order, orchestrate fulfilment, track status
Payments	Payment	Payment, Invoice	Capture payment, store in own database
Identity	Auth	User, Token	Log in, issue and verify signed tokens
Fulfilment	Inventory (designed)	StockItem	Reserve stock (designed, not built in prototype)
Fulfilment	Shipping (designed)	Shipment	Dispatch, tracking (designed, not built in prototype)

The Inventory and Shipping contexts are part of the design but were not built in the prototype; the prototype implements Catalog, Ordering, Payment, and Auth, plus supporting Gateway, Frontend, and Monitoring services.

2. System Architecture

The architecture has a single frontend (UI) service, a set of backend microservices behind an API gateway, and a database per service. The frontend uses widget-based composition. Services communicate over HTTP/REST through the gateway; the order flow is orchestrated by the Ordering service.



Backend network: internal (isolated, no internet). Edge network exposes only Frontend (8000) and Gateway (8080).

Figure 1. Architecture. Solid lines = request-reply over REST through the gateway. Dashed blue = logs and the OrderPaid event sent to the monitoring service. Ordering and Payment each own a separate database (Split Tables).

Components

- **Frontend:** widget-based UI (UI Composition). Independent widgets for Login, Catalog, Recent Orders, and Monitoring, each loading its own data through the gateway.
- **API Gateway:** single entry point; routes requests and verifies the auth token before an order is placed.
- **Backend services:** Auth, Catalog, Ordering (orchestrator), Payment, and Monitoring. Each is autonomous and loosely coupled.
- **Database per service (Split Tables):** Ordering owns `order.db` and Payment owns `payment.db`. No cross-service foreign keys. This is an operational store, not a star or snowflake schema.
- **Isolated network:** backend services share an internal network with no internet access; only the frontend and gateway are exposed to the host.

3. Workflow Pattern

The order process uses **orchestration**. The Ordering service is the orchestrator: it creates the order, calls Payment and checks the reply, and only then marks the order Paid and emits an event. Reactive consumers (the monitoring service) follow a lightweight **choreography** style, simply reacting to events without central control. Authentication is enforced at the gateway: placing an order requires a valid token issued by the Auth service.

4. Implemented Part (Prototype) and Behavior

The implemented slice is the order lifecycle: a logged-in customer places an order, which is paid through the Ordering-to-Payment orchestration and recorded in the databases, with the result logged centrally. The order moves through a simple set of states:

- **AwaitingPayment** — order created, payment not yet confirmed.
- **Paid** — Payment captured the amount and the reply was checked.
- **PaymentFailed / PaymentError** — payment was not captured or the payment call failed.

The implemented extra features are: database per service (Split Tables), widget-based UI, structured logging in every service, a monitoring microservice, authentication, and unit, integration, and end-to-end tests.